

## **1. Explain how you discretize the state-action space and why you think this approach makes sense when it comes to finding a good controller.**

The continuous state of the Mountain Car, which is a pair of numbers (position, velocity), was simplified into just 4 discrete states. This was done by looking at the two most important strategic factors: where the car is and which way it's going. The car's position was categorized based on whether it was to the left or right of the valley's lowest point (which is at position = -0.5).

- Region 1: Left of the valley (position < -0.5)
- Region 2: Right of the valley (position >= -0.5)

The car's velocity was categorized simply by its direction.

- Direction 1: Moving Left (velocity < 0)
- Direction 2: Moving Right (velocity >= 0)

By combining these, we get  $2 \times 2 = 4$  unique states that capture the car's essential situation:

- State 0: On the left hill, moving left (building backward momentum).
- State 1: On the left hill, moving right (starting the climb).
- State 2: On the right hill, moving left (falling back down).
- State 3: On the right hill, moving right (pushing for the goal).

The continuous action (thrust), which can be any number from -1.0 to 1.0, was simplified to just 3 possible actions: [-0.2, 0.0, 0.2].

- -0.2: A gentle push to the left
- 0.0: No push (coasting)
- 0.2: A gentle push to the right

These very low values were chosen because the reward function heavily penalizes large actions ( $-0.1 * \text{action}^2$ ). Using gentle pushes minimizes this penalty, which was the key to achieving a score above 98. This approach makes sense because it is a searchable problem. With 4 states and 3 possible actions for each state, the total number of unique strategies is  $3^4 = 81$ . This is a small enough number to search through exhaustively.

## **2. Why does using the UCB algorithm make sense for this application? What guarantees does the UCB algorithm provide?**

UCB makes sense because by discretizing the problem, we turned it into a "multi-armed bandit" scenario where each of the 81 policies is an "arm." UCB efficiently finds the best policy by intelligently trying new policies and using the policy that works best.

The main guarantee UCB provides is that its regret—the reward lost by having to learn—grows very slowly (logarithmically). This guarantees that the algorithm will converge on the best policy over time and won't get stuck repeatedly choosing a bad one.